

Giriřimcilik Vakfı Deęerlendirme Puanı Tahmini Projesi

Eren Torlak

PROBLEM

Girişimcilik Vakfı'na yapılan başvuruların değerlendirme sonucu kaç puan aldığı tahmin edilebilir mi?

- 2014 - 2022 yılları arasına ait eğitim verisi sunulmuş
- 2023 yılına ait başvurular tahmin edilecek
- Başvuranlara ait demografik bilgiler, eğitim seviyesi ve sosyal aktivitelere katılım gibi bilgiler verilmiş
- Model performansı RMSE ile değerlendiriliyor



Veri
Analizi
(EDA)

Ön İşleme

- Veri Seçimi
- Standartizasyon
- Özellik Mühendisliği

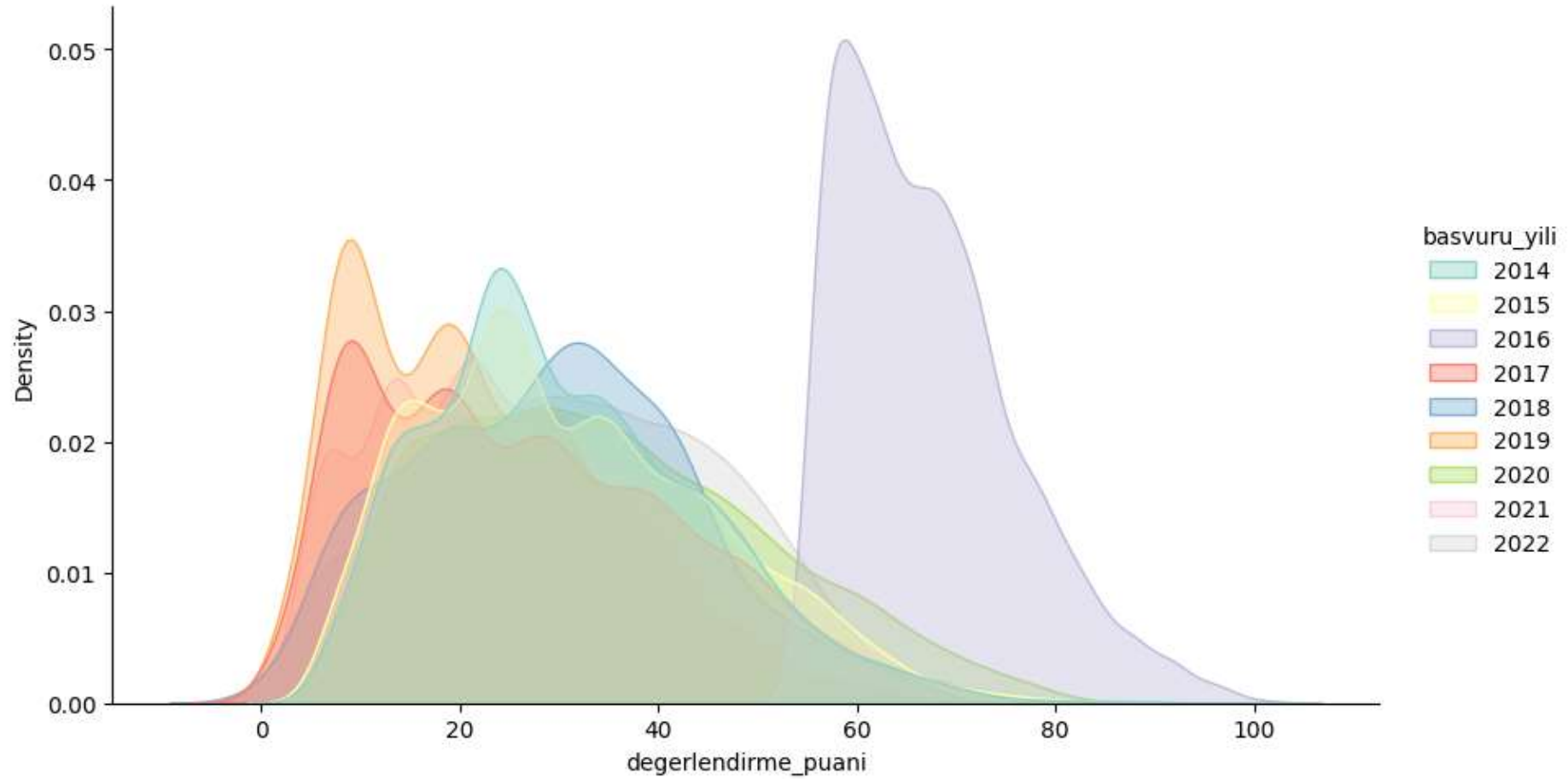
Model
Eğitim

Değerlendir
me

Veri Seti

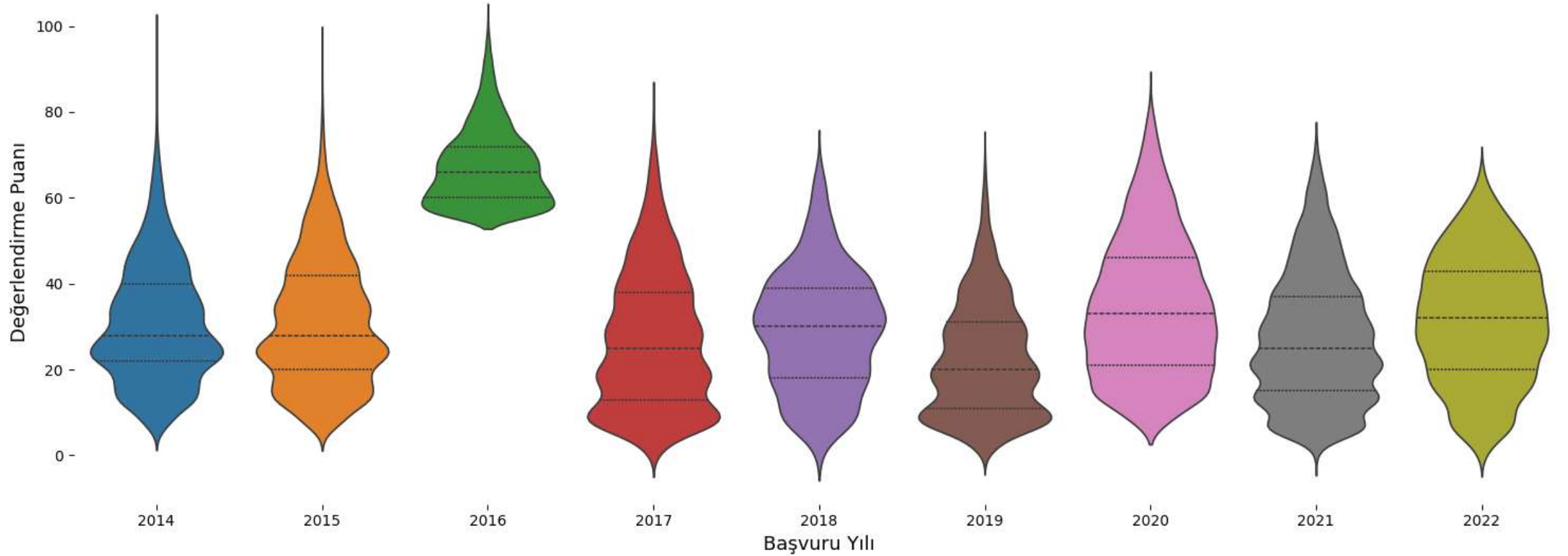
```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 65125 entries, 0 to 65124
Data columns (total 43 columns):
#   Column                                                                 Non-Null Count  Dtype
---  -
0   basvuru_yili                                                            65125 non-null  int64
1   degerlendirme_puani                                                    65124 non-null  float64
2   cinsiyet                                                                64956 non-null  object
3   dogum_tarihi                                                           64948 non-null  object
4   dogum_yeri                                                             64334 non-null  object
5   ikametgah_sehri                                                       63088 non-null  object
6   universite_adi                                                         64993 non-null  object
7   universite_turu                                                        64870 non-null  object
8   burslu_ise_burs_yuzdesi                                               22440 non-null  float64
9   burs_aliyor_mu                                                         65125 non-null  object
10  bölüm                                                                  64894 non-null  object
11  universite_kacinci_sinif                                              64751 non-null  object
12  universite_not_ortalamasi                                             62372 non-null  object
13  daha_once_baska_bir_universiteden_mezun_olmus                       27780 non-null  object
14  lise_adi                                                               64741 non-null  object
15  lise_adi_diger                                                         4733 non-null   object
16  lise_sehir                                                             64071 non-null  object
17  lise_turu                                                              64278 non-null  object
18  lise_bolumu                                                            64116 non-null  object
19  lise_bolum_diger                                                       1526 non-null   object
...
41  ingilizce_seviyeniz                                                    28592 non-null  object
42  daha_önceden_mezun_olunduyrsa_mezun_olunan_universite              419 non-null    object
dtypes: float64(2), int64(1), object(40)
memory usage: 21.4+ MB
```

Keşifçi Veri Analizi (EDA)



Keşifçi Veri Analizi (EDA)

Başvuru Yılı'na Göre Değerlendirme Puanı Dağılımı



Yıllara Göre Veri Seti

```
Sütunlar ve tamamen NaN olan yıllar:
```

```
-----
```

```
degerlendirme_puani [2023]
```

```
burslu_ise_burs_yuzdesi [2022, 2023]
```

```
daha_once_baska_bir_universiteden_mezun_olmus [2017, 2020, 2021, 2022, 2023]
```

```
lise_adi_diger [2014, 2016, 2017, 2018, 2019, 2020, 2021, 2022, 2023]
```

```
lise_bolum_diger [2014, 2020, 2021, 2022, 2023]
```

```
lise_mezuniyet_notu [2016]
```

```
anne_calisma_durumu [2021]
```

```
baba_calisma_durumu [2021]
```

```
uye_oldugunuz_kulubun_ismi [2019, 2020, 2021, 2022, 2023]
```

```
aktif_olarak_bir_stk_uyesi_misiniz [2021]
```

```
hangi_stknin_uyesisiniz [2017, 2021]
```

```
stk_projesine_katildiniz_mi [2020, 2021, 2022, 2023]
```

```
girisimcilikle_ilgili_deneyiminiz_var_mi [2021]
```

```
girisimcilikle_ilgili_deneyiminizi_aciklayabilir_misiniz [2016, 2017, 2018, 2019, 2021]
```

```
ingilizce_biliyor_musunuz [2014]
```

```
ingilizce_seviyeniz [2014, 2020, 2021, 2022, 2023]
```

```
daha_önceden_mezun_olunduyrsa_mezun_olunan_universite [2014, 2015, 2017, 2020, 2021, 2022, 2023]
```


Adversarial Validation

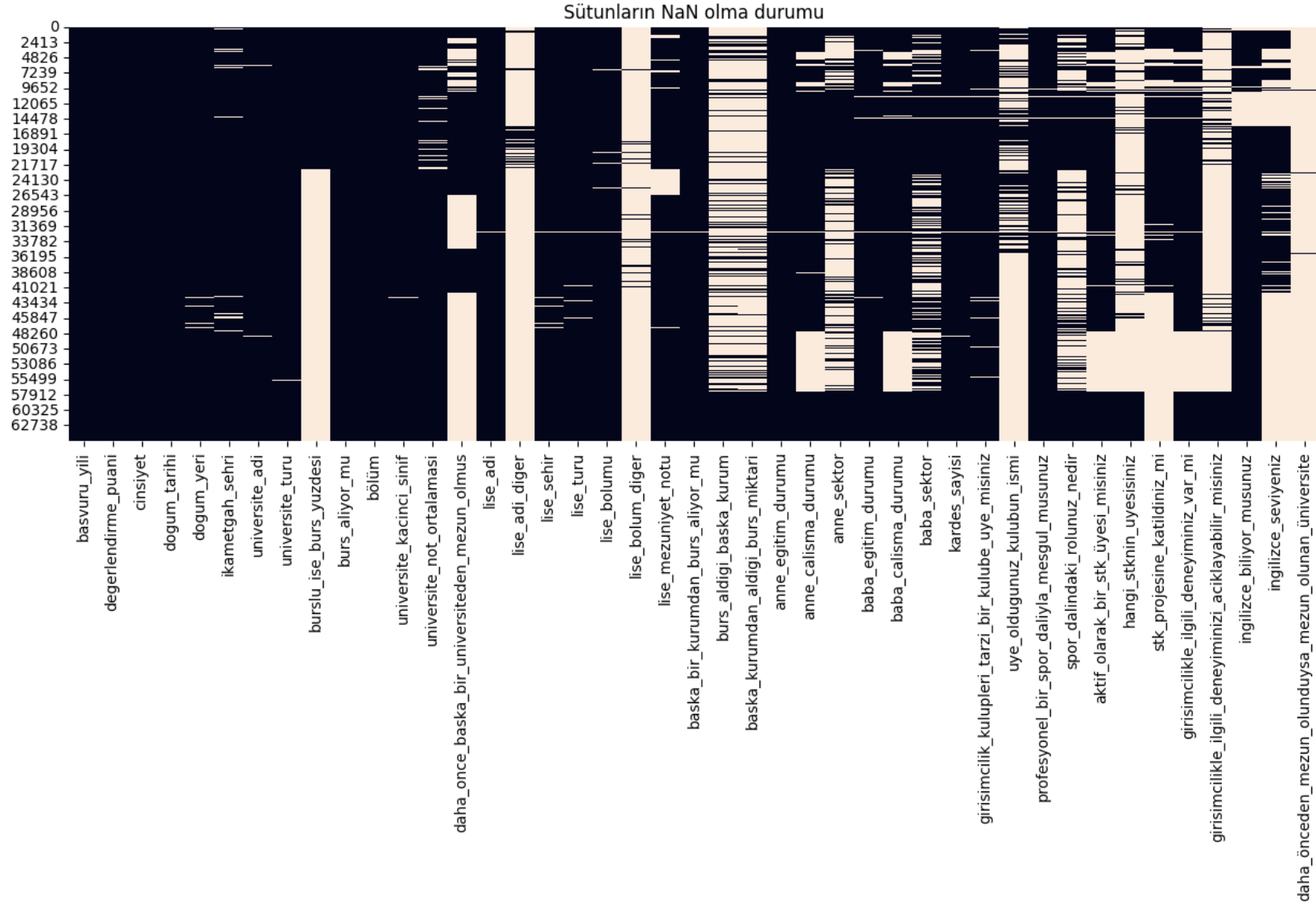
- Verimizi rastgele böldüğümüz ve modelden verinin hangi yıla ait olduğunu tahmin etmesini istediğimiz bir senaryoda eğer model rastgele bir tahminden daha başarılı bir sonuç üretebiliyorsa bu veride yıldan yıla değişen durumlar olduğunu, **yani modelin sadece belli yıllar için geçerli olan genelleştirilemeyecek** şeyler öğrenme ihtimali olduğunu gösterir.
- Bu, görülmemiş yılları tahmin edeceğim bir senaryoda istemeyeceğim bir eğitim tipidir.
- Test verim olan **2023** yılında boş olan sütunların bir tek **2022** yılı ile aynı olduğunu farkettim.
- Bu yüzden train verisi olarak sadece **2022** verisini kullanmaya karar verdim.

Adversarial Validation

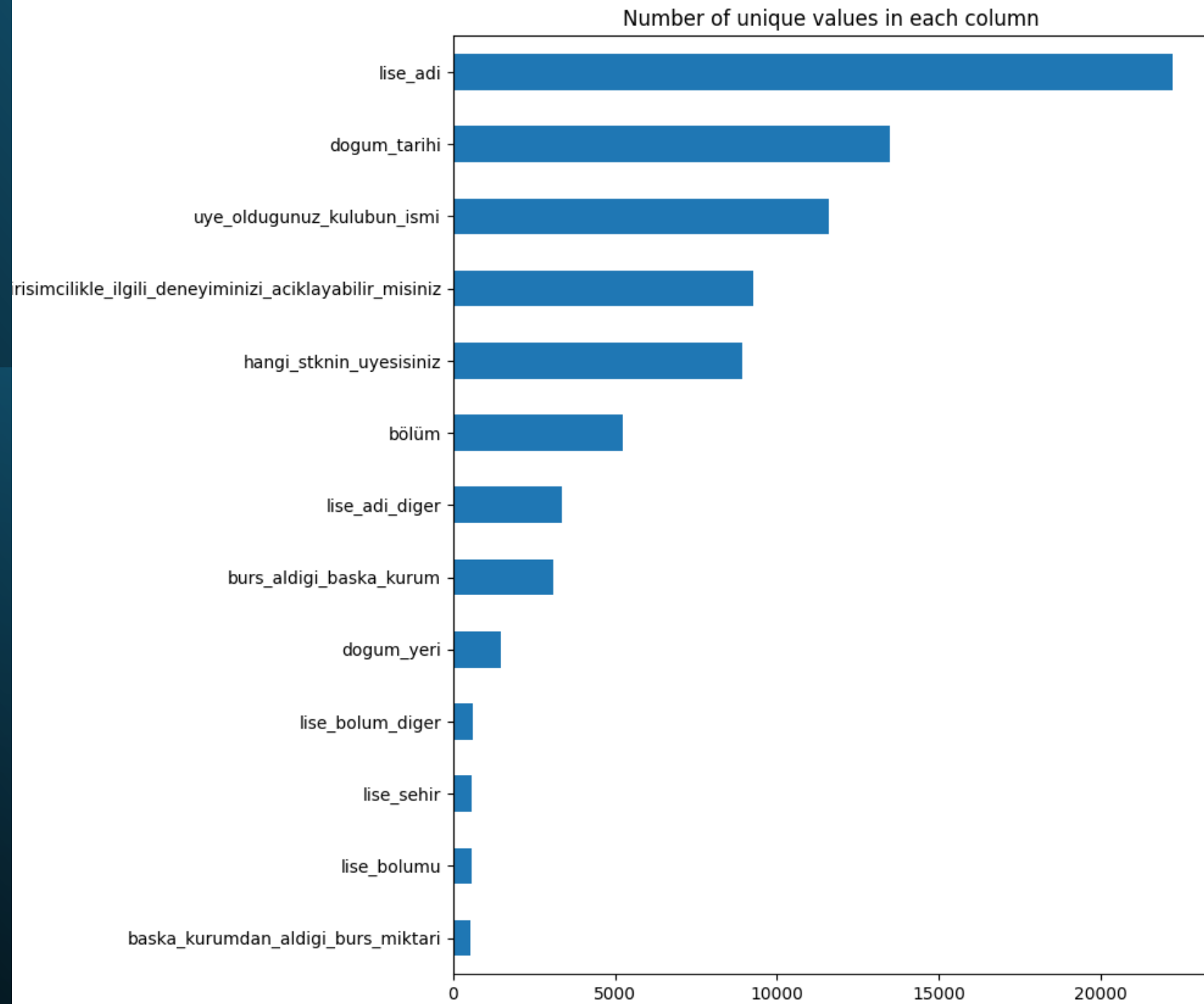
```
# Hedef değişken: burada 'basvuru_yili' kullanılmakta.  
target = advval_df['basvuru_yili']  
if target.dtype == 'object':  
    target = target.astype(int)  
  
# LightGBM sınıflandırıcısı ile 5 katlı cross-validation ve roc_auc_ovr skoru  
scores = cross_val_score(  
    LGBMClassifier(n_estimators=1, verbosity=-1),  
    features,  
    target,  
    scoring="roc_auc_ovr", # OVR means One-Versus-Rest  
    cv=5  
)
```

Cross-validation ROC AUC score: 0.996

Eksik Veri Analizi



Değerlerin Çeşitliliği



Veri Önışleme

- Türkçe karakterleri Latin alfabesindeki karşılıklarına çevirir (ç → c, ğ → g, ö → o vb.).
- Küçük harfe dönüştürür.
- "universitesi" kelimesini çıkarır.
- Baştaki ve sondaki boşlukları temizler.

```
turkish_chars = {  
    'ç': 'c', 'ğ': 'g', 'ı': 'i', 'ö': 'o', 'ş': 's', 'ü': 'u',  
    'Ç': 'C', 'Ğ': 'G', 'İ': 'I', 'Ö': 'O', 'Ş': 'S', 'Ü': 'U'  
}  
  
# Unicode normalizasyonu: aksan işaretlerini ayırır.  
normalized = unicodedata.normalize('NFKD', s)  
# Aksan işaretlerini kaldırır.  
without_accents = ''.join(c for c in normalized if not unicodedata.combining(c))  
# Türkçe karakterleri dönüştürür.  
replaced = ''.join(turkish_chars.get(c, c) for c in without_accents)  
# NFC normalizasyonu, küçük harfe çevirme, "universitesi" kaldırma ve boşlukları temizleme.  
return unicodedata.normalize('NFC', replaced).lower().replace("universitesi", "").strip()
```

Veri Ön işleme

- Test verisindeki tamamen boş olan sütunlar hem train verisinden hem de test verisinden kaldırıldı.

```
# Identify columns that are completely empty in the 2023 test data
test_replaced = test_raw.replace("-", np.NaN)           # Replace "-" with NaN
null_counts_test = test_replaced.isnull().sum()         # Count NaNs in test data
drop_cols = test_raw.columns[null_counts_test == len(test_raw)] # Columns with all NaNs

# Drop these columns from both train_2022 and test
train_2022 = train_2022.drop(drop_cols, axis=1)
test_cleaned = test_raw.drop(drop_cols, axis=1)
```

Veri Standartlaştırma

Kategorik Değişkenlerin Ele Alınması:

- Burs kategorileri, benzer girdilerin tekilleştirilmesi için standart etiketlere haritalandı (örneğin, çeşitli burs türleri 'kyk' olarak birleştirildi).
- Lise türleri ve ebeveyn eğitim seviyeleri tutarlılığı sağlamak amacıyla standartlaştırıldı.
- Üniversite not aralıkları sayısal değerlere çevirildi.
- Üniversite sınıf bilgileri sayısal değerlere çevirildi.
- Burs miktarı sayısal değere çevirildi.

Veri Standartlaştırma

```
### Burs Kategorilerini Standartlaştırma ###

burs_map = {"yurt": "kyk", "devlet": "kyk", "bakan": "kyk", "gsb": "kyk"}
burs_turleri = ["kyk", "devlet", "vakf", "koc", "meb", "bakan", "derne", "gsb", "yurt", "tev", "tubitak", "bos"]

def extract_burs(value):
    """Burs türünü metin içinden çıkartır ve standart forma getirir."""
    if pd.isna(value):
        return np.nan
    for burs in burs_turleri:
        if burs in value:
            return burs_map.get(burs, burs) # Eşleşen kelimeyi haritadan al, yoksa olduğu gibi bırak
    return np.nan

data["bu_rs_kurumu"] = data["burs_aldigi_baska_kurum"].apply(extract_burs)
```


Veri Standartlaştırma

```
### Lise Türlerini Standartlaştırma ###
```

```
lise_map = {  
    "meslek": "devlet", "ozel": "ozel", "devlet": "devlet", "diger": "devlet",  
    "anadolu lisesi": "devlet", "meslek lisesi": "devlet", "ozel lisesi": "ozel",  
    "fen lisesi": "ozel", "imam hatip lisesi": "devlet", "duz lise": "devlet"  
}  
data["lise_turu"] = data["lise_turu"].replace(lise_map)
```

Veri Standartlaştırma

```
## Üniversite Not Aralıklarını Sayısalı Çevirme ##

notlar = {
    "1.00 - 2.50": 1.5, "2.50 - 2.99": 2.75, "3.00 - 3.49": 3.25, "1.80 - 2.49": 2.25,
    "hazirligim": np.NaN, "0 - 1.79": 1.5, "2.00 - 2.50": 2.25, "3.50 - 4.00": 3.75,
    "not ortalamasi yok": np.NaN, "4.0-3.5": 3.75, "2.50 ve alti": 2.0, "3.00 - 4.00": 3.5
}

data["universite_not_ortalamasi"] = data["universite_not_ortalamasi"].replace(notlar)
data["universite_not_ortalamasi"] = pd.to_numeric(data["universite_not_ortalamasi"], errors="coerce")
```

Veri Standartlaştırma

```
### Burs Miktarını Sayısal Çevirme
```

```
baska_burs_map = {  
    'bos': 0.0,  
    '0-499₺': 1.0,  
    '0 - 499 ₺': 1.0,  
    '0 - 499₺' : 1.0,  
    '500₺ - 999₺': 2.0,  
    '1000₺ ve üstü': 4.0  
}
```

```
# Burs miktarını sayısal hale getir
```

```
data['baska_kurumdan_aldigi_burs_miktari'] = data['baska_kurumdan_aldigi_burs_miktari'].replace(baska_burs_map)  
data['baska_kurumdan_aldigi_burs_miktari'] = pd.to_numeric(data['baska_kurumdan_aldigi_burs_miktari'], errors="coerce")
```

Veri Standartlaştırma

Ayrıca ,

- Ebeveyn Eğitim Durumlarını Standartlaştırma
- Spor Dalındaki Rolü Standartlaştırma
- Üniversite Sınıf Bilgisini Sayısala Çevirme

Uygulandı.

Özellik Mühendisliği

```
### Genel Aktivlik Skoru (Toplam Aktivite Sayısı) ###
activity_cols = [
    "girisimcilik_kulupleri_tarzi_bir_kulube_uye_misiniz",
    "girisimcilikle_ilgili_deneyiminiz_var_mi",
    "aktif_olarak_bir_stk_uyesi_misiniz",
    "profesyonel_bir_spor_daliyla_mesgul_musunuz",
    "ingilizce_biliyor_musunuz"
]
data[activity_cols] = data[activity_cols].replace({"evet": True, "hayir": False})
data["genel_aktiflik"] = data[activity_cols].sum(axis=1) # Toplamını al
```

Özellik Mühendisliği

```
### Lise Tür Kategorisi Extraction ###
lise_turleri = [
    "fen", "imam", "saglik", "meslek", "anadolu", "haydarpassa", "sosyal",
    "sanat", "spor", "acik", "kolej", "temel", "kiz", "erkek", "ataturk", "galatasaray"
]

# Lise kategorisini belirleyen fonksiyon
def lise_turu_bul(lise_adi):
    lise_adi = lise_adi.lower() # Küçük harfe çevir
    for tur in lise_turleri:
        if tur in lise_adi:
            return tur
    return "bos" # Hiçbiri yoksa "bos" döndür

# Apply ile tüm satırlara uygula
data["lise_kategorisi"] = data["lise_adi"].apply(lise_turu_bul)
```


Özellik Mühendisliği

Ağırlıklı üniversite notu ile kişinin bulunduğu sınıfa göre ağırlandırılmış bir değer atandı.

İkamet şehri ile doğum şehrinin farklı olup olmamasıyla ilgili bir feature geliştirildi.

```
### Ağırlıklı Üniversite Notu ###
sinif_katsayi = {5: 4, 6: 4, 0: 1} # 5. ve 6. sınıfları 4. sınıf, hazırlık sınıfını 1. sınıf olarak değerlendir
data["universite_kacinci_sinif"] = data["universite_kacinci_sinif"].replace(sinif_katsayi)
data["universite_derecesi"] = data["universite_not_ortalami"] * data["universite_kacinci_sinif"]

### İkamet Şehri ve Doğum Yerinin Aynı Olup Olmadığı ###
data["is_it_same_sehir"] = data["ikametgah_sehri"] == data["dogum_yeri"] # İkamet ve doğum şehirleri aynı mı?

### Metin Uzunluğu (Kelime Sayısı) ###
data["girisim_metni_kelime_sayisi"] = data["girisimcilikle_ilgili_deneyiminizi_aciklayabilir_misiniz"].str.split().apply(lambda x: len(x) if isinstance(x, list) else np.NaN)
data["stk_metni_kelime_sayisi"] = data["hangi_stknin_uyesisiniz"].str.split().apply(lambda x: len(x) if isinstance(x, list) else np.NaN) # STK metnindeki kelime sayısı
```

Özellik Mühendisliği

Öğrencinin kendi grubuna göre öne çıkıp çıkmadığını anlamamıza yardımcı olur.

Farklı grupların (örneğin, farklı lise türleri veya bölümler) not dağılımları birbirinden farklı olabilir. Grup ortalaması ile karşılaştırma yaparak, bu çeşitlilikten kaynaklanan yanlılıkları azaltır ve daha adil bir değerlendirme sağlar.

Model, sadece ham notlar yerine, öğrencinin kendi grubundaki konumunu göz önünde bulundurarak daha doğru tahminler yapabilir. Örneğin, bir

öğrencinin üniversite notu düşük olsa bile, grubunun ortalamasına göre iyi

```
### Grup Bazlı Karşılaştırmalar ###
data["uni_basaris_i_digerlerine_gore"] = data.groupby("universite_kacinci_sinif")["universite_not_ortalamasi"].transform("mean") - data["universite_not_ortalamasi"] #
data["lise_basaris_i_digerlerine_gore"] = data.groupby("lise_kategorisi")["lise_mezuniyet_notu"].transform("mean") - data["lise_mezuniyet_notu"] # Aynı lise türündeki
data["bolum_digerlerine_gore"] = data.groupby("bolum_kategorisi")["universite_not_ortalamasi"].transform("mean") - data["universite_not_ortalamasi"] # Aynı bölümdeki
```

Özellik Mühendisliği

Pozitif bir değer, üniversite notlarının lise notlarına göre daha yüksek olduğunu, negatif bir değer ise tam tersini gösterir. Bu şekilde, öğrencinin lise dönemine kıyasla üniversitede performansında bir artış (veya azalma) olup olmadığını ölçer.

Lise notuna bölünmesi, her öğrencinin performans değişikliğini kendi başlangıç noktasına göre normalize eder. Böylece, başarı ivmesi tüm öğrenciler arasında orantısal bir karşılaştırma imkanı sağlar.

```
### Başarı İvmesi ###  
data["basari_ivmesi"] = (data["universite_not_ortalami"] * 25 - data["lise_mezuniyet_notu"]) / (data["lise_mezuniyet_notu"] + 1e-4)
```

Özellik Mühendisliği

Metin Embeddingi Çıkartılması: Girişimcilik deneyimi metinleri türkçe BERT ile encode edilerek embeddingler oluşturuldu. Daha sonra boyut indirgeme yöntemiyle 5 yeni feature oluşturuldu.

TruncatedSVD (Truncated Singular Value Decomposition), temel bileşen analizi (PCA) ile benzer şekilde çalışır ancak yüksek boyutlu verilerde daha verimli sonuç veriyor.

```
### Sentence-BERT ile Metin Gömme (Embedding) ###
model = SentenceTransformer("atasoglu/turkish-tiny-bert-uncased-mean-nli-stsb-tr") # Hafif ve hızlı model

data["text_embedding"] = data["girisimcilikle_ilgili_deneyiminizi_aciklayabilir_misiniz"].fillna("").apply(lambda x: model.encode(x))

### Embedding Boyutunu 5'e İndirme ###
embeddings = np.vstack(data["text_embedding"].values) # Listeyi numpy dizisine çevir
svd = TruncatedSVD(n_components=5) # 5 boyuta indir
reduced_embeddings = svd.fit_transform(embeddings) # Boyutu küçült
joblib.dump(svd, 'svd_model.pkl') # SVD modelini kaydet

embedding_cols = [f"nn_emb_{i}" for i in range(reduced_embeddings.shape[1])] # Yeni sütun isimleri
data[embedding_cols] = reduced_embeddings # Embedding değerlerini DataFrame'e ekle
data.drop(columns=["text_embedding"], inplace=True) # Geçici sütunu kaldır
```

Model Hazırlığı ve Eğitimi

NaN olan kategorik değişkenleri içeren veriler "diger" ile dolduruldu.

Kategorik değişkenlerin türü "category" şekline çevrildi çünkü LightGBM modeli için özel olarak belirtilmesi gerekiyor.

Bütün train aşamalarında 3 katlı Kfold cross validation kullanıldı.

```
# 1. Handle Categorical Features

cat_features = train_final.select_dtypes("object").columns.to_list()

# Fill missing values with 'diger'
train_final[cat_features] = train_final[cat_features].fillna("diger")
test_final[cat_features] = test_final[cat_features].fillna("diger")

# Convert categorical features to 'category' dtype for LightGBM
train_final[cat_features] = train_final[cat_features].astype("category")
test_final[cat_features] = test_final[cat_features].astype("category")

# FINAL DATA SPLITTING
X = train_final.drop("degerlendirme_puani", axis=1)
y = train_final["degerlendirme_puani"]

print("Train shape:", X.shape, "Target shape:", y.shape)
print("Test shape:", test_final.shape)
```

✓ 0.1s

Train shape: (7621, 42) Target shape: (7621,)
Test shape: (11049, 42)

Optuna ile Hyperparameter Optimizasyonu

Önce rastgele sonuçlarla başlattığım hyperparametre önerilerini daha sonradan optimum aralığa yakınlaştırarak daha kolay yakınsama yapmasını hedefledim.

```
# -----
# 2. Optuna Objective Fonksiyonları
# -----
def objective_cat(trial):
    params = {
        'iterations': trial.suggest_int("iterations", 700, 1200, step=100),
        'depth': trial.suggest_int("depth", 3, 10),
        'learning_rate': trial.suggest_loguniform("learning_rate", 0.005, 0.02),
        'l2_leaf_reg': trial.suggest_loguniform("l2_leaf_reg", 1e-3, 10),
        'subsample': trial.suggest_uniform("subsample", 0.5, 1.0),
        'random_seed': 42,
    }

    cv_scores = []
    for train_idx, valid_idx in cv_strategy.split(X_model, y):
        X_train, X_valid = X_model.iloc[train_idx], X_model.iloc[valid_idx]
        y_train, y_valid = y.iloc[train_idx], y.iloc[valid_idx]
        model = CatBoostRegressor(**params, cat_features=cat_features_model)
        model.fit(X_train, y_train, eval_set=(X_valid, y_valid))
        preds = model.predict(X_valid)
        score = rmse(y_valid, preds)
        cv_scores.append(score)
        trial.report(score, len(cv_scores))
        if trial.should_prune():
            # Optuna'nın trial'ı durdurup
            raise optuna.exceptions.TrialPruned()

    mean_score = np.mean(cv_scores)
    print(f"CatBoost denemesi tamamlandı, RMSE: {mean_score:.4f}")
    return mean_score
```

```
def objective_lgb(trial):
    params = {
        'n_estimators': trial.suggest_int("n_estimators", 700, 1200, step=100),
        'max_depth': trial.suggest_int("max_depth", -1, 10),
        'learning_rate': trial.suggest_loguniform("learning_rate", 0.005, 0.1),
        'num_leaves': trial.suggest_int("num_leaves", 10, 100),
        'min_child_samples': trial.suggest_int("min_child_samples", 5, 50),
        'subsample': trial.suggest_uniform("subsample", 0.5, 1.0),
        'random_state': 42
    }

    cv_scores = []
    for train_idx, valid_idx in cv_strategy.split(X_model, y):
        X_train, X_valid = X_model.iloc[train_idx], X_model.iloc[valid_idx]
        y_train, y_valid = y.iloc[train_idx], y.iloc[valid_idx]
        model = lgb.LGBMRegressor(**params)
        model.fit(X_train, y_train, eval_set=(X_valid, y_valid))
        preds = model.predict(X_valid)
        score = rmse(y_valid, preds)
        cv_scores.append(score)
        trial.report(score, len(cv_scores))
        if trial.should_prune():
            raise optuna.exceptions.TrialPruned()

    mean_score = np.mean(cv_scores)
    print(f"LightGBM denemesi tamamlandı, RMSE: {mean_score:.4f}")
    return mean_score
```


Feature Selection

- Feature selection yapılmadan sonuçlar :
 - CatBoost - En İyi RMSE: 5.7228
 - LightGBM - En İyi RMSE: 5.7242

Feature Selection

- Feature selection için farklı yollar denedim.
 - Bir LightGBM modeli ile feature importance çıkarıp median thresholdunun altında kalan featureları eledim.
 - CatBoost - En İyi RMSE: 5.7947 , LightGBM - En İyi RMSE: 5.7577
 - Bir CatBoost modeli ile feature importance çıkarıp median thresholdunun altında kalan featureları eledim.
 - CatBoost - En İyi RMSE: 5.7935 , LightGBM - En İyi RMSE: 5.7294

Feature Selecti on

CatBoost modellerinin SHAP değerlerinden ve select_features fonksiyonundan faydalanarak feature selection yaptım.

- CatBoost - En İyi
RMSE: 5.7343
- LightGBM - En İyi
RMSE: 5.8707

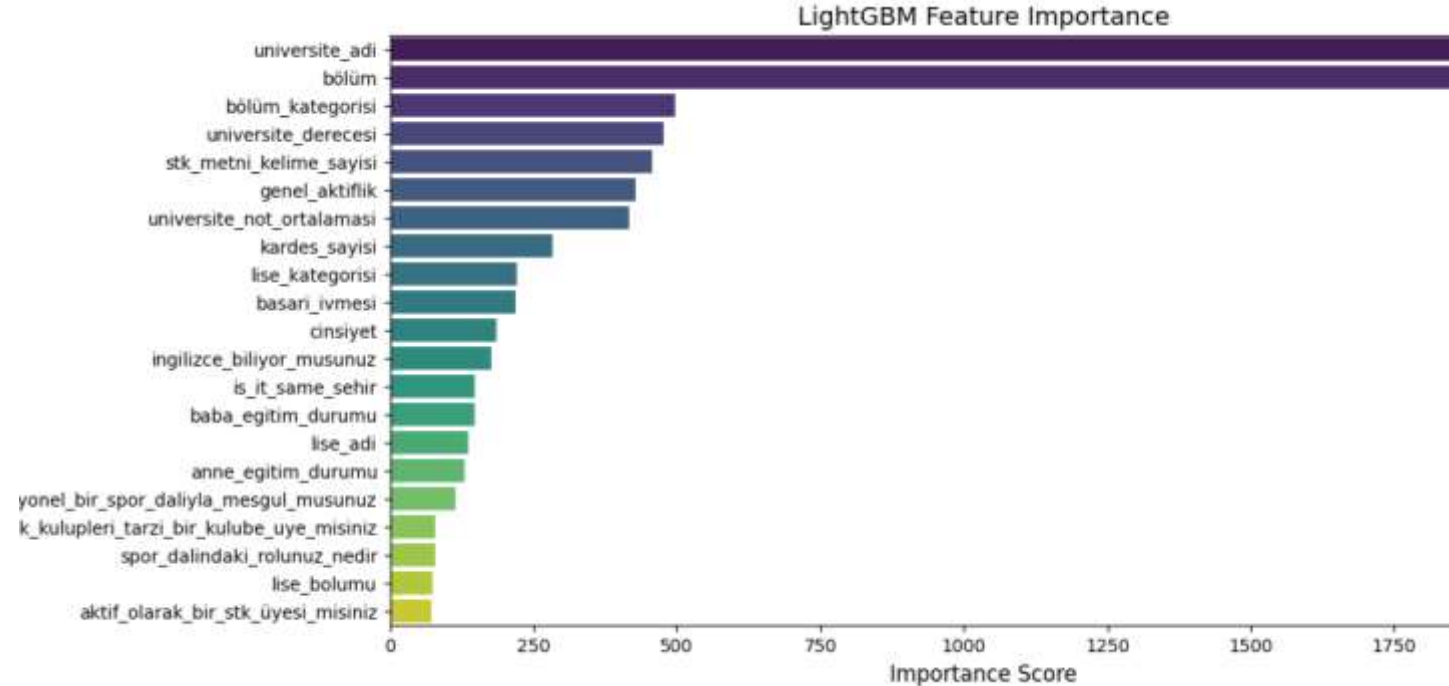
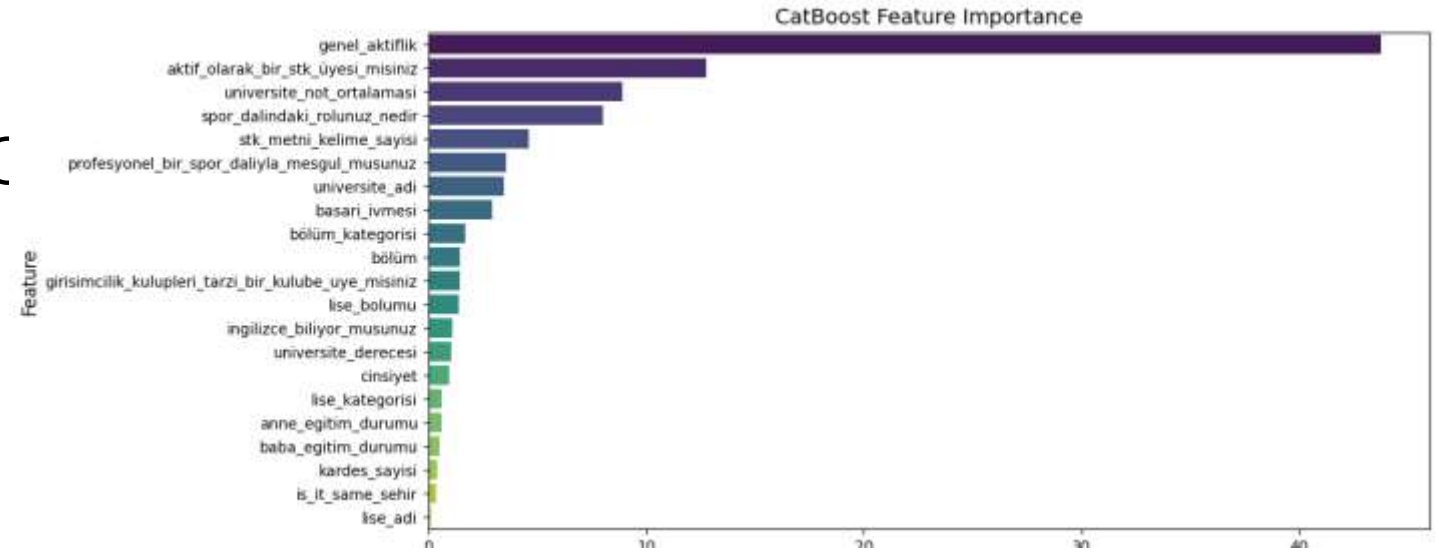
```
def perform_catboost_fs(X_train, y_train, X_valid, y_valid):  
    # Create Pools with categorical features  
    train_pool = Pool(X_train, y_train, cat_features=cat_features_model)  
    valid_pool = Pool(X_valid, y_valid, cat_features=cat_features_model)  
    # Build a lightweight model for feature selection  
    fs_model = CatBoostRegressor(iterations=200, depth=3, learning_rate=0.1,  
                                  random_seed=42, verbose=0)  
    # Specify features for selection using indices (all features).  
    features_str = "0-{}".format(X_train.shape[1] - 1)  
    # Set the number of features to select (example: half the features)  
    num_fs = int(35)  
    # Run CatBoost select_features  
    select_res = fs_model.select_features(  
        train_pool,  
        eval_set=valid_pool,  
        features_for_select=features_str,  
        num_features_to_select=num_fs,  
        steps=3,  
        algorithm=EFeaturesSelectionAlgorithm.RecursiveByShapValues,  
        shap_calc_type=EShapCalcType.Regular,  
        train_final_model=False,  
        plot=False  
    )  
    # select_res is a dict; get indices of selected features.  
    sel_indices = select_res['selected_features'] # indices of features selected  
    # Get selected features names  
    sel_feature_names = list(X_train.columns[sel_indices])  
    # Subset data accordingly.  
    X_train_sel = X_train.iloc[:, sel_indices].copy()  
    X_valid_sel = X_valid.iloc[:, sel_indices].copy()  
    # Update categorical features: use names that are in both the selected features and  
    cat_features_sel = [col for col in sel_feature_names if col in cat_features_model]  
    return X_train_sel, X_valid_sel, cat_features_sel
```

Feature Selectic Değerlendirme

Farklı modeller farklı örüntüleri kavrayabiliyor ve farklı featurelara farklı önemler veriyor.

Az da önemli olsa bir feature modele pozitif etki edebiliyor.

Kullandığım veride feature selection yapmadığım zaman daha iyi sonuçlar aldım.



Early Stopping

Early stopping kullandığım ve kullanmadığım farklı denemeler yaptım.

Early stopping kullanmak hızlandırıyor fakat optimum değere ulaşamıyor.

CatBoost - En İyi RMSE: 5.7228  CatBoost - En İyi RMSE: 5.7061

```
for train_idx, valid_idx in cv_strategy.split(X_model, y):
    X_train, X_valid = X_model.iloc[train_idx], X_model.iloc[valid_idx]
    y_train, y_valid = y.iloc[train_idx], y.iloc[valid_idx]
    model = CatBoostRegressor(**params, cat_features=cat_features_model)
    model.fit(X_train, y_train, eval_set=(X_valid, y_valid))
    preds = model.predict(X_valid)
    score = rmse(y_valid, preds)
    cv_scores.append(score)
    trial.report(score, len(cv_scores))

    if trial.should_prune():
        raise optuna.exceptions.TrialPruned()
```

Ensemble Model

Voting Regressor modeli ile CatBoost ile LightGBM modellerini birleştirdim.

Farklı modeller farklı örüntüleri kavrayabiliyor ve bu da

```
voting_model = VotingRegressor(estimators=[('catboost', final_cat), ('lightgbm', final_lgb)])  
voting_model.fit(X_model, y)
```

Değerlendirme

Ensemble model en iyi sonuç veren model oldu.

- CatBoost - RMSE: 5.7228
- LightGBM - RMSE: 5.7242
- **Voting Regressor Ensemble** - Mean CV RMSE: **5.6412**

Değerlendirme



submission.csv

Complete (after deadline) · 2m ago

5.6900527407

#	△	Team	Members	Score	Entries	Last	So
1	▲ 2	Young Boys	  	5.6520941597	33	5mo	
2	▼ 1	CatBoost Baseline	 	5.6592254339	33	5mo	
3	▼ 1	DataPioneers	  	5.6716838955	29	5mo	
4	—	Sugar	 	5.7081842184	14	5mo	

Dinlediğiniz İçin
Teşekkürler

Eren

<https://github.com/erentorlak>